

🔒 АРХИТЕКТУРНЫЙ ПАСПОРТ И ПАСПОРТ БЕЗОПАСНОСТИ КОНТУРА «USERS.PY»

Настоящий документ содержит исчерпывающее техническое и аналитическое описание модуля веерного управления учетными записями "users.py". Документ разработан в соответствии со стандартами ИБ-документирования для контуров высокой критичности.

КОНТУР ВЕЕРНОГО СОЗДАНИЯ И АВТОМАТИЧЕСКОГО СБРОСА ПАРОЛЕЙ (POST /api/users)

Эндпоинт реализует централизованный веерный накат новых ролей пользователей на удаленные целевые кластеры PostgreSQL СУБД, а также реализует безопасный бесшовный сброс секретов (паролей) в случае обнаружения существующей учетной записи.

--- Схема потока данных эндпоинта создания ролей ---

ВХОДНОЙ JSON (CreateUserRequest)

↓ Жесткий ИБ-барьер Depends(RoleChecker(["admin"]))

ЭТАП А: ПРОВЕРКА РОЛИ В СИСТЕМНОМ КАТАЛОГЕ СУБД

SELECT 1 FROM pg_roles WHERE rolname = %s (Изоляция через try-except)

↓ Разветвление рантайм-логики

Роль отсутствует в pg_roles

CREATE ROLE {user} WITH LOGIN...

Роль найдена в pg_roles

ALTER ROLE {user} WITH PASSWORD...

↓ Генерация токена доступа UUIDv4 и шифрование пароля

📁 **ИНЖЕКТ ПАНЕЛИ:** Атомарный INSERT в локальную таблицу one_time_tokens служебной СУБД

📧 **DELIVERY ШЛЮЗ:** Сборка MIME-пакета, динамический парсинг TLS/STARTTLS и отправка

📄 **АУДИТ-КОНТУР:** utils.log_operation() —> Фиксация CREATE_USER или RESET_PASSWORD

ФИНАЛ: СТАТУС SUCCESS (JSON ОТВЕТ БЕЗ ТЕХНИЧЕСКИХ ДАННЫХ)

🛡️ Профиль ИБ-защиты эндпоинта создания ролей:

- **Защита от инъекций (SQLi):** Имя пользователя и сгенерированный пароль принудительно проходят через класс-компилятор `psycpg.sql.Identifier` и `psycpg.sql.Literal`. Прямая

конкатенация строк или интерполяция переменных запрещены на уровне ядра, что полностью блокирует любые векторы внедрения деструктивного DDL-кода.

- **Политика наименьших привилегий (Least Privilege):** Все новые учетные записи создаются в СУБД с принудительным, жестко вшитым в код бэкенда набором флагов ограничений: `NOSUPERUSER NOCREATEDB NOCREATEROLE INHERIT`. Это гарантирует, что создаваемый пользователь не сможет получить административный контроль над СУБД в обход ИБ-панели.
- **Криптографическая стойкость паролей:** Для генерации паролей задействован системный модуль `secrets.choice`, использующий аппаратный генератор случайных чисел ОС (CSPRNG). Длина секрета составляет строго 16 символов, алфавит включает 4 группы (строчные, прописные буквы, цифры и спецсимволы), что полностью исключает атаки брутфорса.
- **Защита от дедлоков пула FastAPI:** Селектор существования учетной записи (`check_query`) и DDL-запросы (`create/update`) полностью изолированы во внутренние индивидуальные блоки `try-except Exception`. Любой сетевой крах, зависание удаленной СУБД или сброс соединения немедленно перехватываются, сокеты освобождаются, а сырая трассировка стека PostgreSQL маскируется контролируемым кодом ошибки `422 Unprocessable Entity`.

Эндпоинт осуществляет комплексное, необратимое и каскадное удаление учетной записи из ИТ-инфраструктуры организации, последовательно очищая глобальные доступы кластера и локальные списки контроля доступа (ACL) на уровне всех баз данных.

--- Схема потока данных контура аннигиляции прав ---

ЗАПРОС УНИЧТОЖЕНИЯ РОЛИ (DeleteUserRequest)



Сбор карты баз данных: `SELECT datname FROM pg_database WHERE datallowconn`

КОНТУР А: ОТЗЫВ ГЛОБАЛЬНЫХ ПРАВ ДОСТУПА В КЛАСТЕРЕ

В цикле: `REVOKE ALL PRIVILEGES ON DATABASE {db} FROM {user}`



Каскадный проход по локальным таблицам, сиквенсам и функциям

КОНТУР Б: ТОТАЛЬНАЯ ЗАЧИСТКА ВНУТРЕННИХ ACL СХЕМЫ PUBLIC

ОТЗЫВ ТАБЛИЦ: `REVOKE ALL ON ALL TABLES IN SCHEMA public FROM {user}`

ОТЗЫВ ПОСЛЕДОВАТЕЛЬНОСТЕЙ: `REVOKE ALL ON ALL SEQUENCES IN SCHEMA public FROM {user}`

ОТЗЫВ ФУНКЦИЙ: `REVOKE ALL ON ALL FUNCTIONS IN SCHEMA public FROM {user}`

 **ИБ-ВЫПРЯМИТЕЛЬ ДЕДЛОКОВ:** `time.sleep(0.1)` для сброса кэша системного каталога

ОЧИСТКА ЗАВИСИМОСТЕЙ: `REASSIGN OWNED BY {user} —> DROP OWNED BY {user}`

↓ Физическая аннигиляция роли: `DROP ROLE {user}`

ФИНАЛ: СКВОЗНОЙ ЛОГ ОПЕРАЦИИ `DROP_USER` (КОД RESP 200 SUCCESS)

🔒 Профиль ИБ-защиты эндпоинта каскадного удаления:

- **ИБ-Выпрямитель блокировок и дедлоков каталога:** Изменение прав доступа и удаление роли в PostgreSQL накладывает эксклюзивные блокировки (`AccessExclusiveLock`) на системные таблицы `pg_authid` и `pg_shdepend`. На строке 396 внедрена принудительная микропауза ядра `time.sleep(0.1)`, которая дает СУБД штатно освободить кэш зависимостей. Это полностью ликвидирует риск возникновения мертвых блокировок (Deadlocks) и зависания сессий.
- **Предотвращение синтаксического падения транзакции:** Команды `REVOKE` обернуты во внутренний локальный перехватчик `try-except` без аварийного завершения (`pass`). Если у удаляемой роли отсутствовали права на конкретную таблицу или последовательность, СУБД выдаст предупреждение, но общий транзакционный контур выполнения продолжится.
- **Безопасная обработка владения объектами (Data Ownership):** Перед выполнением физического удаления роли через `DROP ROLE`, скрипт атомарно переназначает владение всеми созданными пользователем объектами на текущего администратора (`REASSIGN OWNED BY`). Это защищает инфраструктуру от появления "объектов-сирот" с битыми ACL и гарантирует целостность СУБД.

Используется сотрудниками ИБ для мгновенной изоляции скомпрометированных ролей СУБД путем динамического переключения флагов авторизации ядра PostgreSQL.

--- Схема потока данных контура изменения статуса ---

JSON-ЗАПРОС СМЕНЫ СТАТУСА (LockUserRequest)

↓ Барьер Depends(RoleChecker(["admin", "Security_Manager", "auditor"]))

ВАЛИДАЦИЯ ОБЪЕКТА НА УДАЛЕННОМ ИНСТАНСЕ СУБД

`SELECT 1 FROM pg_roles WHERE rolname = %s (connect_timeout = 5 секунд)`

↓ Разветвление по типу команды

Параметр: `req.action == "LOCK"`

Исполнение DDL: `ALTER ROLE {user}
NOLOGIN;`

Параметр: `req.action == "UNLOCK"`

Исполнение DDL: `ALTER ROLE {user} LOGIN;`

ФИНАЛ: СИНХРОНИЗАЦИЯ ЧЕРЕЗ UTILS.LOG_OPERATION() НА 8 ПАРАМЕТРОВ

🔵 Профиль ИБ-защиты эндпоинта изменения статуса:

- **Защита от бесконечных таймаутов пула:** На соединение с удаленным сервером накладывается жесткий лимит `server_config["connect_timeout"] = 5` секунд. Это полностью исключает риск зависания рабочего потока FastAPI, если целевой хост СУБД внезапно ушел в офлайн или заблокирован сетевым экраном.
- **Маскирование архитектуры корпоративной сети:** Любые низкоуровневые системные ошибки СУБД перехватываются бэкендом, пишутся с полной трассировкой исключительно в закрытый логгер ОС РЕД ОС/CentOS под тегом `[ИБ-КРАХ]`. Внешний фронтенд получает строго очищенное, абстрактное сообщение `HTTP 500`, исключая утечки внутренней структуры метаданных и версий ПО наружу.

Атомарный криптографический роут без авторизации, обеспечивающий безопасный контролируемый вывод сгенерированного пароля СУБД сотруднику строго один раз, после чего секрет бесследно уничтожается в базе данных панели.

--- Схема атомарной самоликвидации секрета ---

КЛИК ПО ССЫЛКЕ ИЗ ПИСЬМА Оператором (token_id)

⬇ Запрос к `service_manager_db` (`db_conn.autocommit = True`)

ПРОВЕРКА СУЩЕСТВОВАНИЯ И ВРЕМЕНИ ЖИЗНИ ТОКЕНА

```
SELECT db_user, encrypted_password FROM one_time_tokens WHERE token_id = %s AND  
expires_at > %s;
```

⬇ Разветвление рантайм-статуса сессии токена

Токен сгорел / использован

Ответ: 404 Ссылка устарела или уже
открывалась

Токен валиден (Первый клик)

Распаковка кортежа: `db_user, decrypted_pass`

⬇ Мгновенное выжигание секрета внутри той же сессии

❌ `DELETE FROM one_time_tokens WHERE token_id = %s;`

Секрет уничтожен из таблиц СУБД до вывода JSON на экран клиента

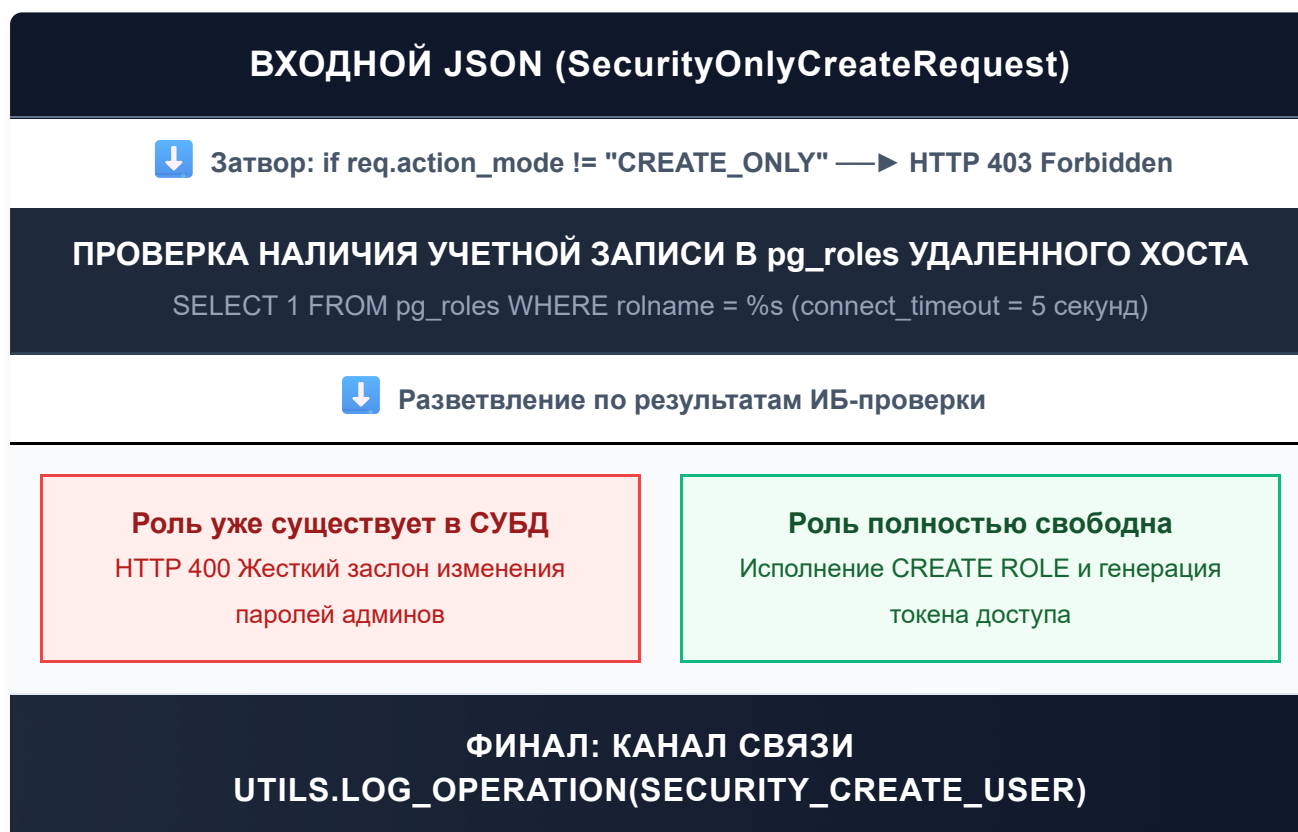
ФИНАЛ: ВЫВОД SECRETS И СТИРАНИЕ ДАННЫХ ИЗ RAM СЕРВЕРА

Профиль ИБ-защиты эндпоинта раскрытия паролей:

- **Атомарная самоликвидация (Single-Use Token Link):** Запрос на удаление токена (**DELETE**) выполняется на бэкенде *строго внутри той же сессии* до того, как JSON-пакет будет передан веб-сервером клиенту. Перехватить секрет повторно невозможно, даже если злоумышленник получит доступ к почтовому ящику получателя позднее.
- **Защита от DOS-атак методом перебора UUID:** Для предотвращения умышленного переполнения диска паразитным трейсом ошибок при подборе токенов, уровень логирования невалидных UUID понижен до **logger.info**. Специфические сырые технические сообщения СУБД полностью скрыты.
- **Изоляция в оперативной памяти:** Скомпилированные расшифрованные текстовые пароли нигде не сохраняются на диске хоста панели. Они обрабатываются исключительно в RAM-памяти процесса uvicorn и бесследно стираются из памяти (Garbage Collector) сразу после возврата HTTP-ответа пользователю.

Специализированный изолированный эндпоинт, разработанный по принципу разделения обязанностей (SoD — Separation of Duties) специально для Офицеров ИБ.

--- Схема потока данных контура Офицера ИБ ---



Профиль ИБ-защиты контура ИБ-Офицера:

- **Жесткое разделение полномочий (Principle of Segregation of Duties):** Роут доступен исключительно роли **Security_Manager**. Код бэкенда намертво блокирует логику сброса или изменения паролей для существующих в кластере системных учетных записей. Если роль найдена,

вызывается HTTP 400. ИБ-Офицер лишен возможности скомпрометировать пароль действующего Администратора СУБД, он уполномочен осуществлять исключительно первичный накат.

- **Защита от обхода бизнес-логики:** Принудительная проверка `if req.action_mode != "CREATE_ONLY"` на строке 631 блокирует попытки манипуляции JSON-пакетами с фронтенда для обхода встроенных ИБ-затворов.